

请记住

- APIs 往往要求访问原始资源 (raw resources)，所以每一个 RAII class 应该提供一个“取得其所管理之资源”的办法。
- 对原始资源的访问可能经由显式转换或隐式转换。一般而言显式转换比较安全，但隐式转换对客户比较方便。

条款 16: 成对使用 new 和 delete 时要采取相同形式

Use the same form in corresponding uses of new and delete.

以下动作有什么错？

```
std::string* stringArray = new std::string[100];  
...  
delete stringArray;
```

每件事看起来都井然有序。使用了 new，也搭配了对应的 delete。但还是有某样东西完全错误：你的程序行为不明确（未有定义）。最低限度，stringArray 所含的 100 个 string 对象中的 99 个不太可能被适当删除，因为它们的析构函数很可能没被调用。

当你使用 new（也就是通过 new 动态生成一个对象），有两件事发生。第一，内存被分配出来（通过名为 operator new 的函数，见条款 49 和条款 51）。第二，针对此内存会有一个（或更多）构造函数被调用。当你使用 delete，也有两件事发生：针对此内存会有一个（或更多）析构函数被调用，然后内存才被释放（通过名为 operator delete 的函数，见条款 51）。delete 的最大问题在于：即将被删除的内存之内究竟存有多少对象？这个问题的答案决定了有多少个析构函数必须被调用起来。

实际上这个问题可以更简单些：即将被删除的那个指针，所指的是单一对象或对象数组？这是个必不可少的问题，因为单一对象的内存布局一般而言不同于数组的内存布局。更明确地说，数组所用的内存通常还包括“数组大小”的记录，以便 delete 知道需要调用多少次析构函数。单一对象的内存则没有这笔记录。你可以把两种不同的内存布局想象如下，其中 n 是数组大小：

单一对象

Object

对象数组

n

Object

Object

Object

...

当然啦，这只是个例子。编译器不需非得这么实现不可，虽然很多编译器的确是这样做的。

当你对着一个指针使用 `delete`，唯一能够让 `delete` 知道内存中是否存在一个“数组大小记录”的办法就是：由你来告诉它。如果你使用 `delete` 时加上中括号（方括号），`delete` 便认定指针指向一个数组，否则它便认定指针指向单一对象：

```
std::string* stringPtr1 = new std::string;
std::string* stringPtr2 = new std::string[100];
...
delete stringPtr1;           //删除一个对象
delete [ ] stringPtr2;       //删除一个由对象组成的数组
```

如果你对 `stringPtr1` 使用 "`delete [ ]`" 形式，会发生什么事？结果未有定义，但不太可能让人愉快。假设内存布局如上，`delete` 会读取若干内存并将它解释为“数组大小”，然后开始多次调用析构函数，浑然不知它所处理的那块内存不但不是个数组，也或许并未持有它正忙着销毁的那种类型的对象。

如果你没有对 `stringPtr2` 使用 "`delete [ ]`" 形式，又会发生什么事呢？唔，其结果亦未有定义，但你可以猜想可能导致太少的析构函数被调用。犹有进者，这对内置类型如 `int` 者亦未有定义（甚至有害），即使这类类型并没有析构函数。

游戏规则很简单：如果你调用 `new` 时使用 `[ ]`，你必须在对应调用 `delete` 时也使用 `[ ]`。如果你调用 `new` 时没有使用 `[ ]`，那么也不该在对应调用 `delete` 时使用 `[ ]`。

当你撰写的 `class` 含有一个指针指向动态分配内存，并提供多个构造函数时，上述规则尤其重要，因为这种情况下你必须小心地在所有构造函数中使用相同形式的 `new` 将指针成员初始化。如果没这样做，又如何知道该在析构函数中使用什么形式的 `delete` 呢？

这个规则对于喜欢使用 `typedef` 的人也很重要，因为它意味 `typedef` 的作者必须说清楚，当程序员以 `new` 创建该种 `typedef` 类型对象时，该以哪一种 `delete` 形式删除之。考虑下面这个 `typedef`：

```
typedef std::string AddressLines[4];           //每个人的地址有 4 行，
                                              //每行是一个 string
```

由于 AddressLines 是个数组，如果这样使用 new:

```
std::string* pal = new AddressLines; //注意, "new AddressLines" 返回
                                   //一个 string*, 就像
                                   // "new string[4]" 一样。
```

那就必须匹配“数组形式”的 delete:

```
delete pal;           //行为未有定义!
delete [ ] pal;       //很好。
```

为避免诸如此类的错误，最好尽量不要对数组形式做 typedefs 动作。这很容易达成，因为 C++ 标准程序库（条款 54）含有 string, vector 等 templates，可将数组的需求降至几乎为零。例如你可以将本例的 AddressLines 定义为“由 strings 组成的一个 vector”，也就是其类型为 vector<string>。

请记住

- 如果你在 new 表达式中使用 []，必须在相应的 delete 表达式中也使用 []。如果你在 new 表达式中不使用 []，一定不要在相应的 delete 表达式中使用 []。

条款 17：以独立语句将 newed 对象置入智能指针

Store newed objects in smart pointers in standalone statements.

假设我们有个函数用来揭示处理程序的优先权，另一个函数用来在某动态分配所得的 Widget 上进行某些带有优先权的处理：

```
int priority();
void processWidget(std::tr1::shared_ptr<Widget> pw, int priority);
```

由于谨记“以对象管理资源”（条款 13）的智慧铭言，processWidget 决定对其动态分配得来的 Widget 运用智能指针（这里采用 tr1::shared_ptr）。

现在考虑调用 processWidget:

```
processWidget(new Widget, priority());
```

等等，不要考虑这个调用形式。它不能通过编译。`tr1::shared_ptr` 构造函数需要一个原始指针（raw pointer），但该构造函数是个 `explicit` 构造函数，无法进行隐式转换，将得自 `"new Widget"` 的原始指针转换为 `processWidget` 所要求的 `tr1::shared_ptr`。如果写成这样就可以通过编译：

```
processWidget(std::tr1::shared_ptr<Widget>(new Widget), priority());
```

令人惊讶的是，虽然我们在此使用“对象管理式资源”（object-managing resources），上述调用却可能泄漏资源。稍后我再详加解释。

编译器产出一个 `processWidget` 调用码之前，必须首先核算即将被传递的各个实参。上述第二实参只是一个单纯的对 `priority` 函数的调用，但第一实参 `std::tr1::shared_ptr<Widget>(new Widget)` 由两部分组成：

- 执行 `"new Widget"` 表达式
- 调用 `tr1::shared_ptr` 构造函数

于是在调用 `processWidget` 之前，编译器必须创建代码，做以下三件事：

- 调用 `priority`
- 执行 `"new Widget"`
- 调用 `tr1::shared_ptr` 构造函数

C++ 编译器以什么样的次序完成这些事情呢？弹性很大。这和其他语言如 Java 和 C# 不同，那两种语言总是以特定次序完成函数参数的核算。可以确定的是 `"new Widget"` 一定执行于 `tr1::shared_ptr` 构造函数被调用之前，因为这个表达式的结果还要被传递作为 `tr1::shared_ptr` 构造函数的一个实参，但对 `priority` 的调用则可以排在第一或第二或第三执行。如果编译器选择以第二顺位执行它（说不定可因此生成更高效的代码，谁知道！），最终获得这样的操作序列：

1. 执行 `"new Widget"`
2. 调用 `priority`
3. 调用 `tr1::shared_ptr` 构造函数

现在请你想想，万一对 `priority` 的调用导致异常，会发生什么事？在此情况下 `"new Widget"` 返回的指针将会遗失，因为它尚未被置入 `tr1::shared_ptr` 内，后者是我们期盼用来防卫资源泄漏的武器。是的，在对 `processWidget` 的调用过程中可能引发资源泄漏，因为在“资源被创建（经由 `"new Widget"`）”和“资源被

转换为资源管理对象”两个时间点之间有可能发生异常干扰。

避免这类问题的办法很简单: 使用分离语句, 分别写出 (1) 创建 Widge, (2) 将它置入一个智能指针内, 然后再把那个智能指针传给 processWidget:

```
std::tr1::shared_ptr<Widget> pw(new Widget);    //在单独语句内以
                                                // 智能指针存储
                                                // newed 所得对象。
processWidget(pw, priority());                  //这个调用动作绝不至于造成泄漏。
```

以上之所以行得通, 因为编译器对于“跨越语句的各项操作”没有重新排列的自由 (只有在语句内它才拥有那个自由度)。在上述修订后的代码内, "newWidget" 表达式以及“对 tr1::shared_ptr 构造函数的调用”这两个动作, 和“对 priority 的调用”是分隔开来的, 位于不同语句内, 所以编译器不得在它们之间任意选择执行次序。

请记住

- 以独立语句将 newed 对象存储于 (置入) 智能指针内。如果不这样做, 一旦异常被抛出, 有可能导致难以察觉的资源泄漏。